

2026 操作系统

1. 生产者消费者问题

1.也称有限缓冲问题，是一个多线程同步问题的经典案例，描述了共享固定大小缓冲区的两个线程在实际运行时会发生的问题

生产者：生产一定量的数据放到缓冲区中，重复此过程

消费者：与此同时消费者也在缓冲区消耗这些数据

问题的关键：保证生产者不会再缓冲区满时加入数据，消费者也不会再缓冲区空时消耗数据

解决：生产者在缓冲区满时休眠，等待下次消费者消耗缓冲区中的数据的时候，生产者才能被唤醒，开始往缓冲区中添加数据

消费者在缓冲区空时休眠，等待生产者往缓冲区添加数据之后，再唤醒消费者

通常采用进程间通信的方法解决该问题

生产者和消费者问题

```
1  #include <iostream>
2  #include <string.h>
3  #include <pthread.h>
4  #include <unistd.h>
5  using namespace std;
6
7  int current = 0; // producer运行加1, consumer运行减1
8  int buf[10];
9  int in = 0, out = 0;
10 int items = 0, spaces = 10;
11 bool flag; // 标记线程结束运行
12 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
13 pthread_cond_t notfull = PTHREAD_COND_INITIALIZER; // 缓冲区不满
14 pthread_cond_t notempty = PTHREAD_COND_INITIALIZER; // 缓冲区不空
15
16 void *producer( void *arg ) {
17     while( flag ) {
18         pthread_mutex_lock( &mutex ); // 为保证条件变量不会因为多线程混乱，所以先加
        锁
19         while( !spaces ) { // 避免“惊群”效应，避免因其他线程实现得到事件而导致该线程
            “假醒”
20             pthread_cond_wait( &notfull, &mutex );
21         }
22         buf[in] = current++;
```

```

23         in = ( in + 1 ) % 10;
24         items++;
25         spaces--;
26
27         printf( "producer %zu , current = %d\n", pthread_self(), current );
28         for( int i = 0; i < 10; i++ ) {
29             printf( "%-4d", buf[i] );
30         }
31         printf( "\n\n" );
32
33         pthread_cond_signal( &notempty );
34         pthread_mutex_unlock( &mutex );
35     }
36     pthread_exit( NULL );
37 }
38
39 void *consumer( void *arg ) {
40     while( flag ) {
41         pthread_mutex_lock( &mutex );
42         while( !items ) {
43             pthread_cond_wait( &notempty, &mutex );
44         }
45         buf[out] = -1;
46         out = ( out + 1 ) % 10;
47         current--;
48         items--;
49         spaces++;
50
51         printf( "consumer %zu , current = %d\n", pthread_self(), current );
52         for( int i = 0; i < 10; i++ ) {
53             printf( "%-4d", buf[i] );
54         }
55         printf( "\n\n" );
56
57         pthread_cond_signal( &notfull );
58         pthread_mutex_unlock( &mutex );
59     }
60     pthread_exit( NULL );
61 }
62
63 int main() {
64     memset( buf, -1, sizeof(buf) );
65     flag = true;
66     pthread_t pro[10], con[10];
67     int i = 0;
68
69     for( int i = 0; i < 10; i++ ) {

```

```

70         pthread_create( &pro[i], NULL, producer, NULL );
71         pthread_create( &con[i], NULL, consumer, NULL );
72     }
73
74     sleep(1); // 让线程运行一秒
75     flag = false;
76
77     for( int i = 0; i < 10; i++ ) {
78         pthread_join( pro[i], NULL );
79         pthread_join( con[i], NULL );
80     }
81
82     return 0;
83 }

```

2. 进程和线程的区别

1.进程是独立的程序运行实例，拥有完整资源空间，是资源分配的基本单位

线程是进程内的执行流，共享进程资源，仅占少量私有资源，是CPU调度的基本单位

2.进程资源占用大（独立的内存空间、寄存器、文件句柄等）

线程占用小（仅栈、程序计数器、寄存器上下文）

3.进程调度开销大（切换需保存 / 恢复整个进程资源，CPU 耗时）

线程开销小（仅切换线程上下文，RTOS 中 μs 级）

4.进程间通信复杂（管道、消息队列、共享内存，需跨进程）

线程间通信简单（同一进程中的线程共享进程内存，嵌入式中直接访问全局变量，但需用互斥锁、信号量避免资源竞争）

5.进程独立性高（进程间互不干扰，一个崩溃不影响其他）

线程独立性低（线程共享进程资源，一个崩溃可能导致整个进程崩溃）

3. 进程间通信

1.管道（匿名管道）

半双工通信，只能用于**有亲缘关系**的进程（父子进程）之间

2.命名管道（FIFO）

可以用于**任意进程**之间通信，但速度慢

3.消息队列

消息按队列存取，支持**不同类型消息**，生命周期随内核，比管道更灵活

容量受系统限制，且信息的复制需要额外消耗CPU时间

4.共享内存

多个进程直接共享同一块内存，**速度最快**，因为不需要数据拷贝；但需要配合**信号量**等做同步

5.信号量

是一个计数器，控制多个进程对共享资源的访问

6.套接字（Socket）

可用于**不同主机、不同设备**之间的进程通信，是网络通信的基础

7.信号

通知接收进程某个事件已经发生，不能用来传递复杂消息